Sprint 06 - Drupal Advanced Development Guide

Stage PFE – Entreprise **VOID**

Stagiaire : Hibat Allah Rguiti

16 mars 2026

Chapitre 1

Day 1 : Work with Data

Drupal repose fortement sur le concept d'entités (*Entities*) pour représenter et gérer les données. Une entité peut représenter différents types d'objets : *nodes*, *users*, médias, etc.

Le système d'entités de Drupal repose sur plusieurs APIs qui permettent de gérer le cycle de vie des données. L'objectif est de comprendre le fonctionnement des principales APIs liées aux entités dans Drupal, notamment : Entity API, EntityQuery, Entity Hooks, Entity Validation API, Entity Access Control et Typed Data API.

1.1. Manipulation des Entités

1.1.1. Entity API

L'**Entity API** constitue le cœur du système de gestion des données dans Drupal. Elle permet de définir et manipuler des entités qui représentent des objets de données structurés.

Une entité peut contenir : des propriétés, des champs (*fields*), des relations avec d'autres entités..

Configuration Entities

Ces entités stockent des configurations du site (ex : types de contenu, rôles, vocabulaires). Elles sont généralement exportables dans des fichiers YAML.

Content Entities

Ces entités contiennent les données réelles du site (ex : *nodes*, utilisateurs, médias). Les développeurs peuvent créer leurs propres types d'entités de contenu personnalisés.

1.1.2. Opérations CRUD sur les entités

Les opérations fondamentales suivent le modèle CRUD (Create, Read, Update, Delete) et sont centralisées via le service `EntityTypeManager`.

1.1.3. Recherche de données avec EntityQuery

Lorsque l'identifiant d'une entité est inconnu, on utilise `EntityQuery` pour filtrer et récupérer des listes d'identifiants d'entités répondant à des critères spécifiques.

```
$query = \Drupal::entityQuery('node')
  ->condition('type', $selected_node->bundle())
  ->condition('nid', $selected_node->id(), '<>')
  ->accessCheck(TRUE);

$nodes = $query->execute();
$other_nodes = $node_storage->loadMultiple($nodes);
```

FIGURE 1.1 – Mécanisme de construction d'une requête EntityQuery

1.1.4. Hooks liés aux entités

Drupal utilise un système de **hooks** pour intervenir à différents moments du cycle de vie des entités.

Hook	**Fonction**
hook_entity_insert	exécuté lors de la création
hook_entity_update	exécuté lors d'une modification
hook_entity_delete	exécuté lors d'une suppression
hook_entity_load	exécuté lors du chargement

Ces hooks permettent d'automatiser des actions comme l'envoi de notifications, la journalisation (logs) ou des traitements de données conditionnels.

1.1.5. Bundle Classes

Dans Drupal, une entité peut être déclinée en plusieurs **bundles** (par exemple, le type de contenu "Article" ou "Page" pour l'entité *Node*). Les **Bundle Classes** permettent d'associer une classe PHP spécifique à chaque bundle pour y encapsuler la logique métier.

1.1.6. Entity Validation API

L'**Entity Validation API** vérifie la conformité des données avant l'enregistrement en s'appuyant sur le composant *Symfony Validator*.

1.1.7. Entity Access Control

Le système **Entity Access Control** définit les permissions granulaires. Chaque entité utilise un `AccessControlHandler` pour valider les droits :

— `checkAccess` : accès aux instances existantes.

— `checkCreateAccess` : droit de création de nouvelles instances.

1.1.8. Typed Data API

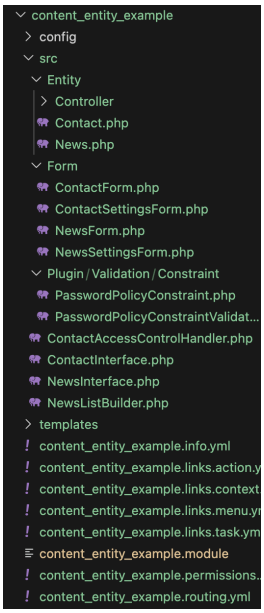
Couche d'abstraction fondamentale, la **Typed Data API** permet de décrire la nature des données (type, définition, contraintes) indépendamment de leur stockage physique.

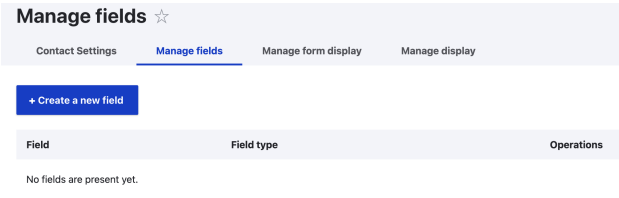
1.2. Création manuelle d'une Content Entity Type

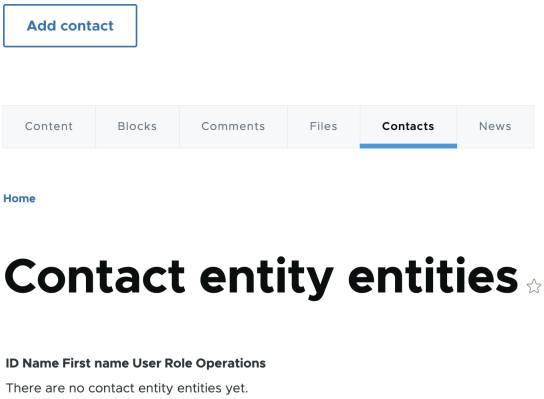
Pour créer une entité personnalisée dans un module :

1. Créer le module `content_entity_example`.
2. Définir la classe de l'entité (`ContentEntityBase`).
3. Ajouter l'attribut `ContentEntityType` (PHP 8) avec ID, table, handlers et liens.
4. Définir les champs de base avec `baseFieldDefinitions()`.
5. Créer les classes `Form` et `ListBuilder`.

La structure est la suivante :







1.3. Génération d'une Content Entity Type avec Drush

Drush permet de générer rapidement les fichiers de base d'une entité personnalisée :

```
void@voids-MacBook-Pro first_module % ./vendor/bin/drush generate entity:content

 Welcome to content-entity generator!


 Module machine name:
 ➤ content_entity_example

 Entity type label [Content Entity Example]:
 ➤ News

 Entity type ID [content_entity_example_news]:
 ➤ news

 Entity class [News]:
 ➤ News

 Entity base path [/news]:
 ➤ /admin/content/news

 Make the entity type fieldable? [Yes]:
 ➤

 Make the entity type revisionable? [No]:
 ➤ yes

 Make the entity type translatable? [No]:
 ➤ yes
```

Cette génération automatique crée :

— La classe de l'entité

— Le formulaire d'édition

— Le ListBuilder

— Les fichiers YAML nécessaires pour activer l'entité

1.4. Questions

1.4.1. Utilisation de `hook_entity_base_field_info_alter` pour appliquer une contrainte sur le champ `pass`

Objectif : Modifier le type d'entité `user` pour appliquer une contrainte de sécurité sur le mot de passe (`pass`) en utilisant le module **Password Policy** de Drupal.

Étapes réalisées :

1. **Installation du module Password Policy :**
   — Le module fournit des outils pour définir des politiques de mot de passe (longueur minimale, complexité, caractères spéciaux, etc.) et un service de validation `password_policy.valid`
   — Configuration d'une politique de mot de passe via l'interface du module.
2. **Création d'une contrainte personnalisée :**
   — Création de `PasswordPolicyConstraint` dans le module custom `content_entity_example`.
   — Cette contrainte est attachée au champ `pass` de l'entité `user` via le hook `hook_entity_base_fiel`

```php
class PasswordPolicyConstraint extends Constraint {

  public $message = 'The password does not respect the password policy.';

}
```

```php
<?php

/**
 * Implements hook_entity_base_field_info_alter().
 */
function content_entity_example_entity_base_field_info_alter(&$fields, \Drupal\Core\Entity\EntityTypeInterface $entity_type) {
  if ($entity_type->id() === 'user' && isset($fields['pass'])) {
    $fields['pass']->addPropertyConstraints('value', [
      'PasswordPolicyConstraint' => [],
    ]);
  }
}
```

3. **Implémentation du Validator :**
   - La classe `PasswordPolicyConstraintValidator` utilise le service `password_policy.validator` injecté pour valider le mot de passe.
   - Le validator récupère la valeur du champ, l'entité associée et exécute la méthode `validatePassword()` du service `PasswordPolicyValidatorInterface`.
   - Si la validation échoue, le validator devrait ajouter une violation avec `buildViolation($message)->addViolation()` pour afficher le message sous le champ.

```php
class PasswordPolicyConstraintValidator extends ConstraintValidator implements ContainerInjectionInterface {

  protected $passwordPolicyValidator;

  public function __construct(PasswordPolicyValidatorInterface $password_policy_validator) {
    $this->passwordPolicyValidator = $password_policy_validator;
  }

  public static function create(ContainerInterface $container) {
    return new static(
      $container->get('password_policy.validator')
    );
  }

  public function validate($value, Constraint $constraint) {
```

4. **Problème rencontré : le message ne s'affiche pas :**
   - Malgré la création du validator et de la contrainte, le message défini dans la contrainte n'apparaît pas sous le champ de mot de passe dans le formulaire.
   - **Cause :** Le module Password Policy gère déjà la validation et son message remplace le message personnalisé. Cependant, les logs montrent que la contrainte est bien appelée.

```
------ -------------- ------------------------ ---------- ---------------------------
 1560   11/Mar 11:01   content_entity_example   Warning    Password policy violation
                                                           during user validation.
 1559   11/Mar 11:01   content_entity_example   Warning    Password policy violation
                                                           during user validation.
```

Remarque : Dans ce cas, il existe déjà un module responsable de la validation du mot de passe, mais la création de contraintes personnalisées peut être utile dans d'autres scénarios.

1.4.2. What is AccessResult and how does it work ?

Dans Drupal, `AccessResult` est une classe utilisée par le système **Access Control** pour déterminer si un utilisateur peut effectuer une action sur une ressource (entité, route, champ, etc.).

Contrairement à un simple booléen (`true` ou `false`), `AccessResult` permet d'exprimer trois états possibles :

- `AccessResult::allowed()` : l'accès est autorisé
- `AccessResult::forbidden()` : l'accès est refusé
- `AccessResult::neutral()` : aucune décision n'est prise

L'état *neutral* permet à Drupal de combiner plusieurs contrôles d'accès. Si un contrôle ne prend pas de décision, un autre contrôle peut toujours déterminer si l'accès doit être autorisé ou refusé.

Cette classe est généralement utilisée dans les méthodes `checkAccess()` et `checkCreateAccess()` des classes `AccessControlHandler` des entités.

```
protected function checkAccess(EntityInterface $entity, $operation, AccountInterface $account) {
  if ($operation == 'delete' && $entity->isNew()) {
    return AccessResult::forbidden()->addCacheableDependency($entity);
  }
  if ($admin_permission = $this->entityType->getAdminPermission()) {
    return AccessResult::allowedIfHasPermission($account, $admin_permission);
  }
  else {
    // No opinion.
    return AccessResult::neutral();
  }
}
```

```
protected function checkCreateAccess(AccountInterface $account, array $context, $entity_bundle = NULL) {
  if ($admin_permission = $this->entityType->getAdminPermission()) {
    return AccessResult::allowedIfHasPermission($account, $admin_permission);
  }
  else {
    // No opinion.
    return AccessResult::neutral();
  }
}
```

1.4.3. Creating a custom entity quickly

La création d'une entité personnalisée dans Drupal nécessite plusieurs fichiers et classes, ce qui implique beaucoup de *boilerplate code*.

Pour simplifier ce processus, Drupal fournit des générateurs via l'outil en ligne de commande **Drush** :

```
drush generate entity:content
```

Cette commande pose plusieurs questions (nom du module, nom de l'entité, table de base de données, clés de l'entité, etc.) puis génère automatiquement la structure de base du code. Cela permet d'accélérer la création d'entités personnalisées et d'éviter d'écrire manuellement tout le code de base.

1.4.4. How do you get a field definition using code ?

Dans Drupal, une *field definition* représente la définition structurelle d'un champ d'une entité. Pour récupérer la définition d'un champ par code, on peut utiliser le service `entity_field.manager`.

Exemple :

```
$fields = \Drupal::service('entity_field.manager')
  ->getFieldDefinitions('node', 'article');

$title_definition = $fields['title'];
```

Dans cet exemple, la méthode `getFieldDefinitions()` retourne la liste des définitions de champs pour l'entité `node` du bundle `article`. On peut ensuite accéder à la définition d'un champ spécifique.

1.4.5. Is it possible to create multiple field formatters for a field type ? If so, how ?

Oui, il est possible de créer plusieurs **formatters** pour un même type de champ. Un **formatter** définit comment les données du champ seront affichées sur le site.

On peut créer un **formatter personnalisé** en déclarant un plugin de type `FieldFormatter`.

```php
namespace Drupal\mymodule\Plugin\Field\FieldFormatter;

use Drupal\Core\Field\FormatterBase;
use Drupal\Core\Field\FieldItemListInterface;

/**
 * @FieldFormatter(
 *   id = "custom_text_formatter",
 *   label = @Translation("Custom Text Formatter"),
 *   field_types = {"text"}
 * )
 */
class CustomTextFormatter extends FormatterBase {
  public function viewElements(FieldItemListInterface $items, $langcode) {
    $elements = [];
    foreach ($items as $delta => $item) {
      $elements[$delta] = ['#markup' => strtoupper($item->value)];
    }
    return $elements;
  }
}
```

Après avoir créé ce plugin, le nouveau formatter apparaît automatiquement dans *Manage display* pour tous les champs du type `text`.

1.4.6. Using Drush, how to retrieve a module configuration (settings) ?

Pour récupérer la configuration d'un module avec Drush, on utilise la commande `drush cget` (*config :get*) suivie du nom de la configuration.

```
void@voids-MacBook-Pro first_module % ./vendor/bin/drush cget system.site
_core:
  default_config_hash: ijfbzDTN4CbE7Sr-6ubWzy_t1vH4OtU1doNCLssVz-4
langcode: en
uuid: 7cb2f96c-951e-434b-b434-5401b5c0b2ba
name: 'Module Creation'
mail: hiba@gmail.com
slogan: ''
page:
  403: ''
  404: ''
  front: /weather
admin_compact_mode: false
weight_select_max: 100
default_langcode: en
mail_notification: null

void@voids-MacBook-Pro first_module % ./vendor/bin/drush cget system.site name
'system.site:name': 'Module Creation'

void@voids-MacBook-Pro first_module % ./vendor/bin/drush cget anytown.settings
display_forecast: 1
location: '12321'
weather_closures: "Ice rink closed until winter.\r\nParking behind Apple Lane is closed."

void@voids-MacBook-Pro first_module % ./vendor/bin/drush cget anytown.settings location
'anytown.settings:location': '12321'
```

Chapitre 2

Day 2 : Work with Hooks

2.1. Define and Invoke a New Hook

Lorsqu'un module effectue une action spécifique (par exemple ajouter un contenu personnalisé, gérer des données, etc.), il peut être nécessaire d'offrir aux autres développeurs un point d'extension. Un nouveau hook permet à d'autres modules de s'accrocher au code existant sans modifier le module initial. Cela rend le module plus flexible et extensible.

2.1.1. Trois étapes pour créer un nouveau hook

Pour définir et invoquer un hook personnalisé dans un module, les étapes suivantes doivent être respectées :

Créer un nom unique

Le nom doit être choisi de manière à ne pas entrer en conflit avec les autres hooks. Habituellement, le nom est préfixé par celui du module.

Documenter le hook

Dans un fichier spécial `monmodule.api.php`, il convient d'écrire une description du hook comprenant :

— ce qu'il fait
— ses paramètres
— ce qu'il doit retourner (si nécessaire)

Cela permet aux autres développeurs de comprendre comment l'utiliser.

Appeler (invoker) le hook dans le code

L'appel au hook doit être placé à un moment logique dans le module, en utilisant le service `module_handler`. Exemple avec `invokeAll()` :

```
$results = \Drupal::service('module_handler')->invokeAll('monmodule_do_something',
```

```
[$arg1, $arg2]);
```

Cette instruction indique à Drupal : *« Appelle toutes les implémentations de ce hook dans les modules activés et retourne leurs résultats. »*

2.2. Questions

2.2.1. How do you add a new base field to an existing entity type using `hook_entity_base_field_info` ?

Le hook `hook_entity_base_field_info()` permet de définir des **base fields** pour un type d'entité.

Ce hook est exécuté lorsque Drupal construit la définition des entités, notamment lors de l'installation du module, sa mise à jour du module et la reconstruction du cache des entités.

```
function content_entity_example_entity_base_field_info(EntityTypeInterface $entity_type) {

  if ($entity_type->id() === 'node') {

    $fields['example_field'] = BaseFieldDefinition::create('string')
      ->setLabel(t('Example field'))
      ->setDescription(t('Example base field'));

    return $fields;
  }

  return [];
}
```

Ainsi, si un nouveau base field est ajouté dans ce hook après l'installation du module, il ne sera réellement créé dans la base de données qu'après une **mise à jour du module**.

Dans ce cas, il est nécessaire d'exécuter une mise à jour de base de données (par exemple avec `drush updatedb`) pour appliquer les modifications.

2.2.2. What is the role of `hook_update_N` ?

Le hook `hook_update_N()` permet de mettre à jour la base de données ou la configuration d'un module lors d'une nouvelle version.

Il est exécuté automatiquement lors de l'exécution de :

— la commande `drush updatedb`,

— ou via l'interface `/update.php`.

Numérotation des hooks Le numéro `N` suit une convention structurée :

— Premier chiffre(s) : version majeure de Drupal supportée.

— Chiffre(s) suivants : version majeure du module.

— Dernier(s) chiffre(s) : compteur séquentiel de l'update hook pour cette version du module.

Exemple : `8111`

— 8 → Drupal 8

— 1 → version majeure du module 1.x

— 11 → 11e update hook pour cette version du module

```php
/**
 * Adds a field to node in hook_update_N().
 */
function content_entity_example_update_9100() {

  if (!FieldStorageConfig::loadByName('node', 'field_example')) {
    FieldStorageConfig::create([
      'field_name' => 'field_example',
      'entity_type' => 'node',
      'type' => 'string',
      'cardinality' => 1,
    ])->save();
  }

  if (!FieldConfig::loadByName('node', 'article', 'field_example')) {
    FieldConfig::create([
      'field_name' => 'field_example',
      'entity_type' => 'node',
      'bundle' => 'article',
      'label' => t('Example field'),
      'required' => FALSE,
    ])->save();
  }
}
```

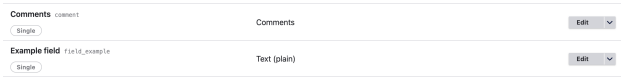
Dans cet exemple, le hook `content_entity_example_update_9100()` est utilisé pour ajouter un nouveau champ `field_example` aux nœuds de type `article`.

Ainsi, cet update hook permet d'ajouter automatiquement un nouveau champ aux articles lors de la mise à jour du module.

```
void@voids-MacBook-Pro first_module % ./vendor/bin/drush updatedb
 ------------------------ ----------- --------------- -------------------------
  Module                   Update ID   Type            Description
 ------------------------ ----------- --------------- -------------------------
  content_entity_example   9100        hook_update_n   9100 - Adds a field to
                                                       node in
                                                       hook_update_N().
 ------------------------ ----------- --------------- -------------------------

 Do you wish to run the specified pending updates?
 Yes

 >  [notice] Update started: content_entity_example_update_9100
 >  [notice] Update completed: content_entity_example_update_9100
  [success] Finished performing updates.
void@voids-MacBook-Pro first_module % ./vendor/bin/drush cr
  [success] Cache rebuild complete.
 void@voids-MacBook-Pro first_module %
```



2.2.3. What is the role of `hook_install` ?

Le hook `hook_install()` est exécuté lors de l'activation (installation) d'un module.

Il permet notamment de :

— créer les tables ou champs nécessaires au module,

— ajouter des configurations par défaut,

— initialiser certaines données requises pour le fonctionnement du module.

Ce hook est exécuté une seule fois, au moment de l'installation du module.

2.2.4. How would you prefix all your newly created nodes (type : article) with HEY- using `hook_ENTITY_TYPE_presave` ?

Le hook `hook_ENTITY_TYPE_presave()` est exécuté juste avant l'enregistrement d'une entité dans la base de données.

Il permet de modifier les données de l'entité avant qu'elles ne soient sauvegardées.

```php
/**
 * Implements hook_ENTITY_TYPE_presave() for node entities.
 */
function content_entity_example_node_presave(NodeInterface $node) {
  if ($node->bundle() === 'article') {
    $prefix = 'HEY-';
    if (strpos($node->getTitle(), $prefix) !== 0) {
      $node->setTitle($prefix . $node->getTitle());
    }
  }
}
```

Title	Content type	Author	Status	Updated	Language	Operations
HEY-Article1	Article	admin	Published	13 Mar 2026 - 14:10	English	Edit

Dans ce cas, ce hook est utilisé pour vérifier si l'entité est de type `article` et modifier automatiquement son titre en ajoutant un préfixe `HEY-` avant l'enregistrement.

2.2.5. What is the role of `$entity->original` ?

La propriété `$entity->original` contient la version originale de l'entité telle qu'elle était avant sa modification.

Elle est principalement utilisée lors de la mise à jour d'une entité afin de comparer l'état précédent et le nouvel état de l'entité.

Cela permet par exemple de :

— détecter si un champ a été modifié,

— exécuter une logique spécifique uniquement lorsque certaines valeurs changent.

2.2.6. How can you override a Theme Hook provided by another module ?

Pour surcharger un *Theme Hook* fourni par un autre module, plusieurs hooks peuvent être utilisés.

`hook_theme_registry_alter()` permet de modifier le registre des thèmes de Drupal. Ce hook permet de changer le template ou la fonction utilisée pour le rendu d'un hook de thème.

`hook_preprocess_HOOK()` permet de modifier les variables envoyées au template Twig avant que celui-ci ne soit rendu. Il est souvent utilisé pour ajouter ou transformer des variables utilisées dans l'affichage.

```php
/**
 * Preprocess theme variables for templates.
 *
 * This hook allows modules to preprocess theme variables for theme templates.
 * hook_preprocess_HOOK() can be used to preprocess variables for a specific
 * theme hook.
 *
 * For more detailed information, see the
 * @link themeable Theme system overview topic @endlink.
 *
 * @param array $variables
 *   The variables array (modify in place).
 * @param string $hook
 *   The name of the theme hook.
 */
function hook_preprocess(&$variables, $hook): void {
  static $hooks;
```

```php
function hook_theme_registry_alter(&$theme_registry) {
  // Kill the next/previous my_module topic navigation links.
  foreach ($theme_registry['my_module_topic_navigation']['preprocess functions'] as $key => $value) {
    if ($value == 'template_preprocess_my_module_topic_navigation') {
      unset($theme_registry['my_module_topic_navigation']['preprocess functions'][$key]);
    }
  }
}
```

2.2.7. Using `hook_theme_suggestions_alter` how can you add a new theme suggestion for `$hook === 'user'` based on the view mode ?

Le hook `hook_theme_suggestions_alter()` permet de modifier ou d'ajouter des suggestions de templates Twig avant que Drupal ne choisisse celui qui sera utilisé pour le rendu.

Il est possible d'ajouter une suggestion basée sur certaines conditions, par exemple le *view mode* d'une entité.

Dans le cas du hook `node`, on peut vérifier le *view mode* courant et ajouter une suggestion de template spécifique.

Par exemple, si le view mode est `compact`, Drupal pourra utiliser un template comme :

Cette approche permet de créer différents templates pour un même type d'entité selon le mode d'affichage utilisé.

Chapitre 3

Day 3 : Work with configuration & features

3.1. Practice

L'objectif de est de comprendre comment gérer la configuration dans Drupal et d'utiliser différents outils pour synchroniser et déployer la configuration entre un environnement de développement (DEV) et un environnement de production (PROD). Les méthodes utilisées sont :

— Export/Import standard de configuration
— Config Split pour séparer les modules de développement
— Features pour encapsuler une configuration spécifique dans un module réutilisable

3.1.1. Mise en place des environnements

Nous avons configuré deux instances Drupal sur la machine :

— **DEV :** environnement de développement
— **PROD :** environnement de production

Dans DEV, nous avons :

— Installé le module `devel` via Composer et activé
— Créé un content type **News** avec des champs (Body, Image, Category)
— Créé une taxonomie **Categories**
— Créé un rôle **Content Manager** avec permissions pour gérer le content type

3.1.2. Export et import de la configuration

Étapes

1. Export de la configuration DEV :

```
void@192 drupal_dev % ./vendor/bin/drush cex -y
 [success] Configuration successfully exported to ../config/sync.
../config/sync
```

```
drupal_dev
  config / sync
    announcements_feed.settings.yml
    automated_cron.settings.yml
    block_content.type.basic.yml
    block.block.claro_breadcrumbs.yml
    block.block.claro_content.yml
    block.block.claro_help_search.yml
    block.block.claro_help.yml
    block.block.claro_local_actions.yml
    block.block.claro_messages.yml
    block.block.claro_page_title.yml
    block.block.claro_primary_local_tasks.yml
    block.block.claro_secondary_local_tasks.yml
    block.block.olivero_account_menu.yml
    block.block.olivero_breadcrumbs.yml
    block.block.olivero_content.yml
    block.block.olivero_help.yml
    block.block.olivero_main_menu.yml
    block.block.olivero_messages.yml
    block.block.olivero_page_title.yml
    block.block.olivero_powered.yml
    block.block.olivero_primary_admin_actions.yml
    block.block.olivero_primary_local_tasks.yml
    block.block.olivero_search_form_narrow.yml
    block.block.olivero_search_form_wide.yml
    block.block.olivero_secondary_local_tasks.yml
    block.block.olivero_site_branding.yml
    claro.settings.yml
    coffee.configuration.yml
```

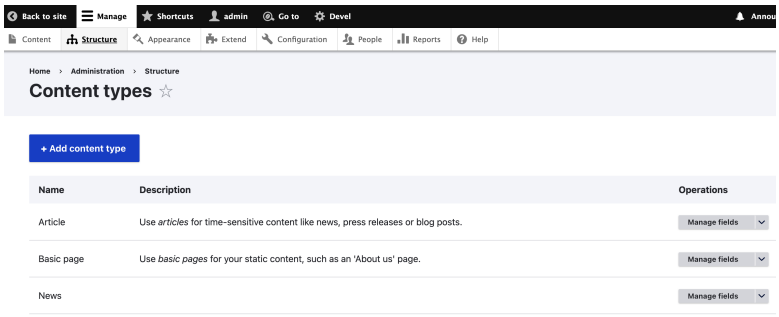
2. Import de la configuration en PROD :

```
 [notice] Synchronized configuration: create views.view.content.
 [notice] Synchronized configuration: create views.view.content_recent.
 [notice] Synchronized configuration: create views.view.files.
 [notice] Synchronized configuration: create views.view.frontpage.
 [notice] Synchronized configuration: create views.view.glossary.
 [notice] Synchronized configuration: create views.view.taxonomy_term.
 [notice] Synchronized configuration: create views.view.user_admin_people.
 [notice] Synchronized configuration: create views.view.watchdog.
 [notice] Synchronized configuration: create views.view.who_s_new.
 [notice] Synchronized configuration: create views.view.who_s_online.
 [notice] Synchronized configuration: update system.site.
 [notice] Finalizing configuration synchronization.
 [success] The configuration was imported successfully.
void@192 drupal_prod %
```

Résultats et observations

— Après l'import, tout le content type, champs, taxonomie et rôle sont présents

— **Devel est également installé** en PROD, car la liste des modules activés est incluse dans la configuration (`core.extension.yml`)



Réponse à la question : Was Devel installed in production ? **Réponse :** Oui, Devel a été installé en production car il faisait partie de la configuration exportée.

3.1.3. Utilisation de Config Split

Objectif

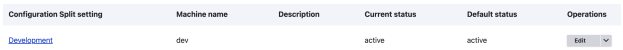
Exclure les modules de développement (Devel...) de la configuration destinée à la production.

Étapes

1. Installer Config Split en DEV :

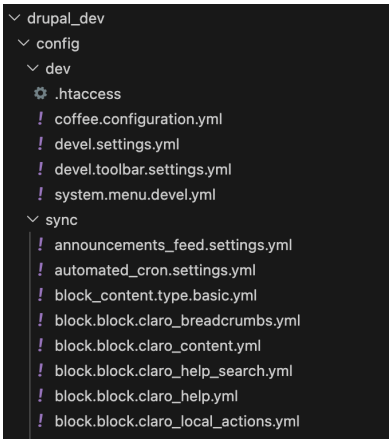
```
void@192 drupal_dev % composer require drupal/config_split

./composer.json has been updated
Running composer update drupal/config_split
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking drupal/config_split (2.0.2)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Downloading drupal/config_split (2.0.2)
  - Installing drupal/config_split (2.0.2): Extracting archive
2 package suggestions were added by new dependencies, use `composer suggest` t
Package pear/console_getopt is abandoned, you should avoid using it. No replac
Generating autoload files
47 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
Using version ^2.0 for drupal/config_split
void@192 drupal_dev % ./vendor/bin/drush en config_split -y
 [success] Module config_split has been installed. (Permissions - Configure)
```



2. Exporter la configuration DEV :

```
drush cex -y
```



3. Importer vers PROD :

```
void@192 drupal_prod % ./vendor/bin/drush cim -y
+------------+-------------------------------------------+------------+
| Collection | Config                                    | Operation  |
+------------+-------------------------------------------+------------+
|            | config_split.config_split.dev             | Create     |
|            | core.extension                            | Update     |
|            | search.settings                           | Update     |
|            | core.entity_view_display.user.user.default | Update     |
|            | core.entity_view_display.user.user.compact | Update     |
|            | coffee.configuration                      | Delete     |
|            | system.menu.devel                         | Delete     |
|            | devel.toolbar.settings                    | Delete     |
|            | devel.settings                            | Delete     |
+------------+-------------------------------------------+------------+

 // Import the listed configuration changes?: yes.

 [notice] Synchronized extensions: uninstalled devel.
 [notice] Synchronized extensions: uninstalled coffee.
 [notice] Synchronized extensions: installed config_split.
 [notice] Synchronized configuration: create config_split.config_split.dev.
 [notice] Synchronized configuration: update search.settings.
 [notice] Synchronized configuration: update core.entity_view_display.user.user.default.
 [notice] Synchronized configuration: update core.entity_view_display.user.user.compact.
 [notice] Finalizing configuration synchronization.
 [success] The configuration was imported successfully.
```

Résultats

- Les modules de développement (Devel...) ne sont plus activés en production
- La configuration utile (content type, taxonomie, rôles) est correctement appliquée

3.1.4. Utilisation de Features

Objectif

Créer un module encapsulant uniquement la configuration nécessaire (content type, champs, taxonomie, rôles) pour faciliter la réutilisation et les mises à jour successives.

Étapes

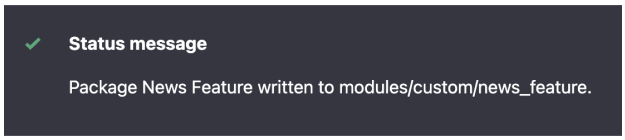
1. Installer Features en DEV :

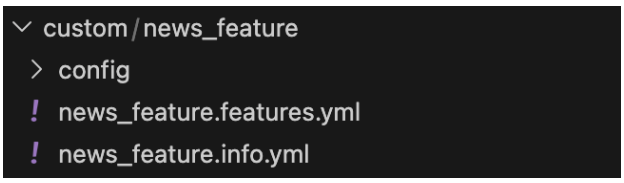
```
void@192 drupal_dev % composer require 'drupal/features:^3.16'
./composer.json has been updated
Running composer update drupal/features
Loading composer repositories with package information
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
  - Locking drupal/config_update (2.0.0-alpha4)
  - Locking drupal/features (3.16.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
  - Installing drupal/config_update (2.0.0-alpha4): Extracting archive
  - Installing drupal/features (3.16.0): Extracting archive
Package pear/console_getopt is abandoned, you should avoid using it. No replacement was suggested.
Generating autoload files
47 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
void@192 drupal_dev % ./vendor/bin/drush en features -y
The following module(s) will be installed: features, config_update

 // Do you want to continue?: yes.

 [success] Module features has been installed.
 [success] Module config_update has been installed.
void@192 drupal_dev %
```

2. Créer une Feature `News Feature` :
   - Inclure le content type **News** et tous ses champs
   - Inclure la taxonomie **Categories**
   - Inclure le rôle **Content Manager**



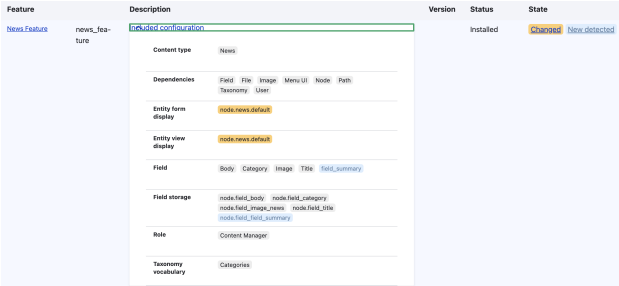


3. Activer la Feature en PROD :

```
void@192 drupal_prod % ./vendor/bin/drush en news_feature -y

 [success] Module news_feature has been installed.
void@192 drupal_prod % ./vendor/bin/drush cr
 [success] Cache rebuild complete.
void@192 drupal_prod %
```

4. Modifier le content type en PROD (changement de nom)
5. Ajouter un nouveau champ en DEV, mettre à jour la Feature, copier et re-déployer en PROD :




```
void@192 drupal_prod % ./vendor/bin/drush features:import news_feature -y

 // Do you really want to import news_feature : field.storage.node.field_field_summary?: yes.

 // Do you really want to import news_feature : field.field.node.news.field_field_summary?: yes.

 // Do you really want to import news_feature : core.entity_form_display.node.news.default?: yes.

 // Do you really want to import news_feature : core.entity_view_display.node.news.default?: yes.

void@192 drupal_prod % ./vendor/bin/drush cr
 [success] Cache rebuild complete.
void@192 drupal_prod %
```

Résultats et observations

— Après un revert, la Feature applique les modifications du module
— Les changements manuels en PROD (comme le nom du content type) peuvent être écrasés

Réponse à la question : Y a-t-il eu des conflits lors de l'import de la Feature ? **Réponse :** Oui, si la configuration existait déjà en PROD. Dans notre cas, la modification du nom du content type a été écrasée par la configuration de la Feature.

Chapitre 4

Day 4 : Work with Cache

4.1. Cache en Drupal

Drupal utilise un système de cache complexe pour améliorer les performances et réduire les temps de réponse. Plusieurs types de cache existent :

4.1.1. Types de cache principaux

- **Page Cache** : cache la page entière pour les utilisateurs anonymes afin de servir rapidement les contenus statiques.
- **Render Cache** : cache les render arrays afin de ne pas recalculer l'affichage des entités ou blocs à chaque requête.
- **Dynamic Page Cache** : cache les parties de page pour tous les utilisateurs (même authentifiés), mais en tenant compte des contextes de cache (ex : rôles, permissions).
- **Cache API** : permet aux développeurs de stocker en cache des données arbitraires.

4.2. Practice

4.2.1. Objectif

Créer une route `/api/articles` qui renvoie 3 articles (nodes) en JSON avec les champs `nid` et `title`. L'exercice permet de tester :

- l'application de **Max-age** sur les réponses,
- l'utilisation des **cache tags**,
- et l'invalidation automatique du cache lorsque les données changent.

4.2.2. Étapes réalisées

1. **Création de la route et du controller** :
   - Déclaration d'une route `/api/articles` dans `content_entity_example.routing.yml`.

— Création du controller correspondant pour récupérer les nodes via `entityQuery()` et construire la réponse JSON.

```yaml
content_entity_example.api_articles:
  path: '/api/articles'
  defaults:
    _controller: '\Drupal\content_entity_example\Controller\ArticlesController::listArticles'
    _title: 'Articles API'
  requirements:
    _permission: 'access content'
```

```php
class ArticlesController extends ControllerBase {

  public function listArticles() {

    $nids = [1, 2, 3];
    $data = [];

    $nodes = Node::loadMultiple($nids);

    foreach ($nodes as $node) {
      if ($node->bundle() === 'article') {
        $data[] = [
          'nid' => $node->id(),
          'title' => $node->getTitle(),
        ];
      }
    }

    $response = new CacheableJsonResponse($data);

    $response->getCacheableMetadata()->setCacheMaxAge(3600);

    return $response;

  }

}
```

2. **Test initial avec Max-age** :
   — Nous avons utilisé `CacheableJsonResponse` et défini un `setMaxAge(3600)`.

   Max-age : durée de validité du cache en secondes. Après ce délai, le cache devient périmé et doit être recalculé.

   — Résultat : le titre modifié d'un node s'affiche immédiatement malgré le Max-age.
   — Analyse : le problème venait de `settings.local.php` où les caches étaient désactivés :
   — Solution : nous avons commenté ces lignes pour réactiver les caches.

```php
<?php

// $settings['cache']['bins']['render'] = 'cache.backend.null';
// $settings['cache']['bins']['dynamic_page_cache'] = 'cache.backend.null';
// $settings['cache']['bins']['page'] = 'cache.backend.null';
// Désactive l'affichage des en-têtes X-Drupal-Cache-* (UNCACHEABLE, etc.).
// Décommenter la ligne ci-dessous pour charger development.services.yml (debug cache).
$settings['container_yamls'][] = DRUPAL_ROOT . '/sites/development.services.yml';
```

▾ Response headers ☐ Raw

Cache-Control	must-revalidate, no-cache, private
Connection	Keep-Alive
Content-Language	en
Content-Length	108
Content-Type	application/json
Date	Mon, 16 Mar 2026 05:28:47 GMT
Expires	Sun, 19 Nov 1978 05:00:00 GMT
Keep-Alive	timeout=5, max=100
Server	Apache/2.4.66 (Unix) PHP/8.5.3
X-Content-Type-Options	nosniff
X-Drupal-Cache	UNCACHEABLE (request policy)
X-Drupal-Cache-Contexts	user.permissions
X-Drupal-Cache-Max-Age	3600
X-Drupal-Cache-Tags	http_response
X-Drupal-Dynamic-Cache	MISS
X-Frame-Options	SAMEORIGIN
X-Generator	Drupal 11 (https://www.drupal.org)
X-Powered-By	PHP/8.5.3

En observant les headers HTTP, on voit que le `Max-age` est appliqué, c'est-à-dire que même si l'on modifie le contenu des nodes, le cache reste valide pendant la durée définie avant d'être recalculé.

3. **Ajout des cache tags** :

   Cache tags : identifie des éléments de contenu (ex : node :1, node :2) pour que Drupal sache quels caches invalider lorsque ces entités changent.

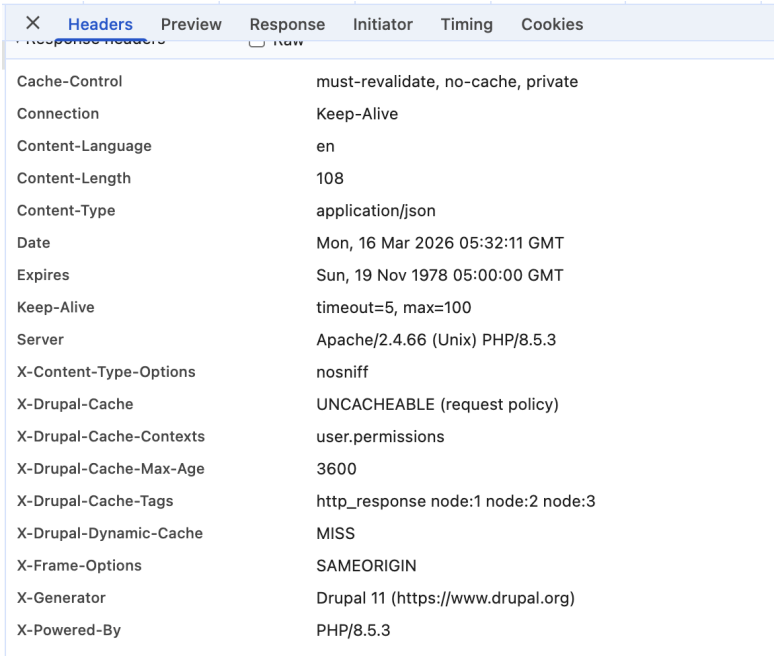
```php
$response = new CacheableJsonResponse($data);

$response->getCacheableMetadata()->setCacheMaxAge(3600);
$metadata = new \Drupal\Core\Cache\CacheableMetadata();
$metadata->setCacheTags(['node:1','node:2','node:3']);

$response->addCacheableDependency($metadata);

return $response;
```

4. **Test après cache tags** :
   - Après modification du titre d'un des nodes inclus dans les tags, la réponse JSON se met à jour instantanément.
   - Les headers HTTP montrent maintenant :
     - `X-Drupal-Cache-Tags: node:1 node:2 node:3`
     - `X-Drupal-Cache-Max-Age: 3600`



Headers Preview Response Initiator Timing Cookies

Cache-Control	must-revalidate, no-cache, private
Connection	Keep-Alive
Content-Language	en
Content-Length	108
Content-Type	application/json
Date	Mon, 16 Mar 2026 05:32:11 GMT
Expires	Sun, 19 Nov 1978 05:00:00 GMT
Keep-Alive	timeout=5, max=100
Server	Apache/2.4.66 (Unix) PHP/8.5.3
X-Content-Type-Options	nosniff
X-Drupal-Cache	UNCACHEABLE (request policy)
X-Drupal-Cache-Contexts	user.permissions
X-Drupal-Cache-Max-Age	3600
X-Drupal-Cache-Tags	http_response node:1 node:2 node:3
X-Drupal-Dynamic-Cache	MISS
X-Frame-Options	SAMEORIGIN
X-Generator	Drupal 11 (https://www.drupal.org)
X-Powered-By	PHP/8.5.3

5. **Vérification du debug** :
   — Avec `http.response.debug_cacheability_headers: true` activé dans `development.service`
   nous avons pu inspecter toutes les métadonnées de cache.

Chapitre 5

Day 5 : Work with Data Migration

La migration de données dans Drupal consiste à importer des données depuis une source externe vers le site Drupal tout en permettant de transformer ces données avant leur insertion. L'exemple suivant illustre le processus avec un fichier CSV représentant des voitures.

5.1. Exemple : Migration d'un fichier CSV "Cars"

5.1.1. Fichier source CSV

Le fichier CSV contient les informations suivantes :

first_module > web > modules > custom > content_entity_example > migration_files > cars.csv

```csv
ID,Make,Model,Year
1,Toyota,Corolla,2020
2,Honda,Civic,2019
3,Ford,Focus,2021
```

5.1.2. Type de contenu Drupal

Un type de contenu **Car** a été créé avec :

— **Title** : champ nœud standard, pour le nom complet de la voiture

— **field_year** : nombre entier pour l'année

5.1.3. Migration YAML

Le fichier de migration YAML utilisé est :

```yaml
id: cars_migration
label: 'Migrate Cars from CSV'

source:
  plugin: csv
  path: 'modules/custom/content_entity_example/migration_files/cars.csv'
  header_offset: 0
  ids:
    - ID
  fields:
    - name: ID
      label: 'Unique ID'
    - name: Make
      label: 'Car brand'
    - name: Model
      label: 'Car model'
    - name: Year
      label: 'Manufacture year'

process:
  title:
    plugin: concat
    source:
      - Make
      - Model
    delimiter: ' '
  field_year: Year

destination:
  plugin: 'entity:node'
  default_bundle: car
```

5.1.4. Explication

— **Source :** le plugin `csv` lit le fichier CSV. Le champ `ID` est l'identifiant unique pour chaque ligne.

— **Process :** le champ `title` du nœud Drupal est généré en concaténant `Make` et `Model`, et `field_year` récupère directement `Year`.

— **Destination :** chaque ligne devient un nœud Drupal du type `Car`.

```
void@192 first_module % ./vendor/bin/drush migrate-status
 ------------------- ---------------- -------- ------- ---------- -------------
---------------- ----------------
  Group               Migration ID     Status   Total   Imported   Unprocessed
 Message Count   Last Imported
 ------------------- ---------------- -------- ------- ---------- -------------
---------------- ----------------
  Default (default)   cars_migration   Idle     3       0          3
 0
 ------------------- ---------------- -------- ------- ---------- -------------
```

```
void@192 first_module % ./vendor/bin/drush migrate-import cars_migration
 [notice] Processed 3 items (3 created, 0 updated, 0 failed, 0 ignored) - done w
ith 'cars_migration'
void@192 first_module %
```

	Title	Content type
☐	Toyota Corolla	Car
☐	Honda Civic	Car
☐	Ford Focus	Car

5.2. Questions spécifiques sur Drupal Migration API

5.2.1. What's the role of `migration_lookup` ?

Le plugin `migration_lookup` permet de **créer des relations entre les entités migrées**. Par exemple, si on migre des articles liés à des auteurs déjà migrés, `migration_lookup` permet de récupérer l'ID Drupal de l'auteur correspondant afin de l'associer correctement à l'article.

5.2.2. Import depuis une autre source que CSV, par exemple MySQL

Pour importer depuis une base MySQL :

— Utiliser un **plugin source SQL**, comme `dsql` ou `sql`.
— Définir la **connexion à la base MySQL** dans `settings.php`.
— Fournir la requête SQL pour récupérer les données.
— Le reste de la migration (process et destination) reste identique à un CSV.

5.2.3. Rollback d'une migration

Pour annuler une migration déjà exécutée :

— Utiliser Drush :

```
drush migrate-rollback <migration_id>
```

— Drupal supprime toutes les entités créées par cette migration.
— Utile avant de corriger ou de relancer la migration.

5.2.4. Traitement d'un champ avant import

Pour transformer un champ avant de l'importer, on utilise un **process plugin**.

Exemple : tronquer un titre à 10 caractères :

```
process:
  title:
    plugin: callback
    callable: substr
    source: Title
    arguments:
      - 0
      - 10
```

- Ici, la fonction PHP `substr` est appliquée sur la valeur source pour limiter la longueur à 10 caractères.

Lien vers le dépôt GitHub

Le code source complet de ce sprint est disponible sur mon dépôt GitHub :

https://github.com/Hibat-Allah-RGUITI/Custom_entity_type.git